

Modul 4

State Hoisting, Perubahan Konfigurasi, dan Side-effect

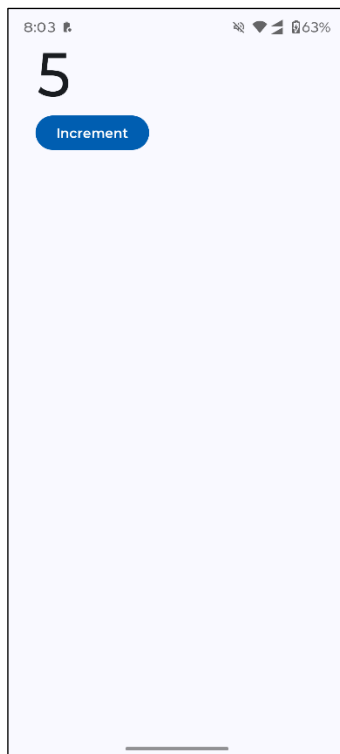
Muhammad Rasyid Ridho
245150600111003

Link GitHub: <https://github.com/raysyid/TicketApp2.git>

1. State Hoisting

Aktivitas Percobaan

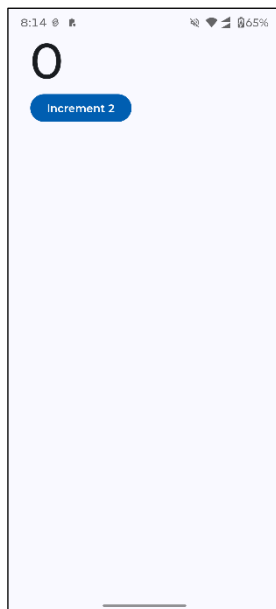
- 1) Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.



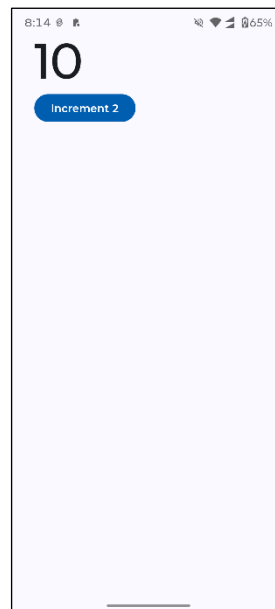
- a. Apa yang terjadi setiap kali tombol "Increment" diklik?
 - Setiap kali tombol Increment diklik, nilai angka yang ditampilkan pada Text bertambah 1. Misalnya dari 0 menjadi 1, kemudian 2, 3, dan seterusnya.
- b. Apa yang menyebabkan hal tersebut terjadi?
 - Hal tersebut terjadi karena pada Button terdapat event onClick yang menjalankan kode: `onClick = { number++ }` Ketika tombol diklik, nilai variabel number bertambah satu. Karena number merupakan state Compose, perubahan nilainya menyebabkan recomposition, sehingga Text menampilkan nilai terbaru.
- c. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?

- Variabel state yang berperan adalah: `var number by remember { mutableStateOf(0) }` Variabel number digunakan oleh Text melalui: `Text(text = "$number", fontSize = 72.sp)`
Jadi, ketika nilai number berubah, Text akan menampilkan nilai terbaru.
 - d. Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?
 - Variabel state yang digunakan adalah: `val label by remember { mutableStateOf("Increment") }` Variabel tersebut digunakan pada: `Text(text = "$label")` Nilai awal label adalah "Increment", sehingga tombol menampilkan tulisan Increment.
- 2) Pada percobaan tahap sebelumnya, ubah kode program pada Composable `CounterScreen` agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".
- a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?
 - Perubahan dilakukan di dalam fungsi Composable `CounterScreen`, yaitu:
 - Mengubah nilai label dari `"Increment"` menjadi `"Increment 2"`
 - Mengubah event `onClick` dari `number++` menjadi `number += 2`
 - Dengan demikian, setiap tombol diklik nilai number akan bertambah 2 dan label tombol berubah menjadi Increment 2.
 - b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

Sebelum diklik:



Setelah diklik 5 kali:



- 3) Langkah selanjutnya adalah melakukan *state* dan *event hoisting* pada state dan event yang sebelumnya terdapat di dalam Composable `CounterScreen`.
Ubah signature kode program pada bagian fungsi Composable `CounterScreen` dari yang sebelumnya:

```
fun CounterScreen(modifier: Modifier)
```

menjadi:

```
fun CounterScreen(modifier: Modifier,
                  number: Int, label: String, onClick: () ->
Unit)
```

Ubah body di dalam Composable CounterScreen menjadi:

```
// var number by remember { mutableStateOf(0) }
// val label by remember { mutableStateOf("Increment") }
Column(modifier) {
    Text(text = "$number", fontSize = 72.sp)
    Button(onClick = onClick) {
        Text(text = "$label")
    }
}
```

Ubah pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() dari yang sebelumnya:

```
CounterScreen(modifier = Modifier.padding(32.dp))
```

menjadi:

```
var number by remember { mutableStateOf(0) }
CounterScreen(Modifier.padding(32.dp),
    number = number,
    label = "Increment",
    onClick = { number++ }
)
```

Perhatikan bahwa deklarasi variabel state number dari yang sebelumnya dilakukan di dalam fungsi Composable CounterScreen, sekarang dideklarasikan di dalam fungsi onCreate() sebelum fungsi Composable CounterScreen dipanggil dari dalam fungsi onCreate().

- 4) Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.
 - a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?
 - Ya, program tetap berjalan dengan baik dan menghasilkan tampilan serta perilaku yang sama. Tombol Increment tetap dapat diklik dan nilai number tetap bertambah satu setiap kali tombol diklik. Perbedaannya terletak pada lokasi pengelolaan state dan event, bukan pada tampilan atau hasil interaksi pengguna.
 - b. Di bagian kode program manakah saat ini proses increment dilakukan?
 - Proses increment sekarang dilakukan pada lambda onClick ketika CounterScreen dipanggil di dalam onCreate(): `onClick = { number++ }`

- c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?
- State number sekarang dideklarasikan di dalam onCreate(): `var number by remember { mutableStateOf(0) }` Sedangkan CounterScreen hanya menerima number dan label melalui parameter:
`number: Int,`
`label: String`
Text dan Button sendiri tetap didefinisikan di dalam CounterScreen, tetapi state-nya dikelola oleh pemanggil CounterScreen.

5) Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:

- Label tombol "Increment" menjadi "Double"
- Nilai awal variabel number dari 0 menjadi 1.
- Ubah fungsi/lambda onClick pada pemanggilan Composable CounterScreen di dalam fungsi onCreate() sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel number.
- Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

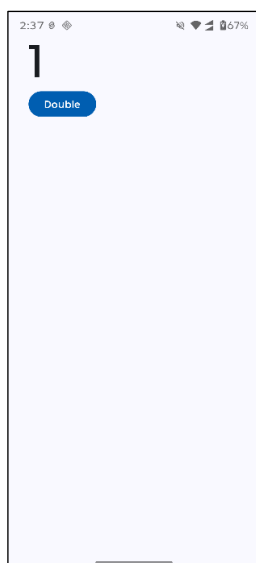
```
var number by remember { mutableStateOf(1) }
```

```
CounterScreen(  
    (Modifier.padding(32.dp),  
    number = number,  
    label = "Double",  
    onClick = { number *= 2 }  
)
```

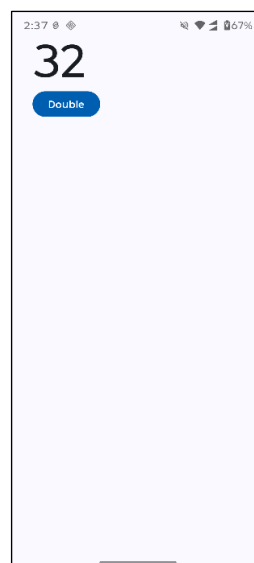
6) Jalankan program,

- Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.

Sebelum



Setelah 5 kali di-klik



- b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?
- Perubahan dilakukan pada pemanggilan CounterScreen di dalam fungsi onCreate(), khususnya pada:

```
var number by remember { mutableStateOf(1) }  
label = "Double"
```

dan:

```
onButtonClick = { number *= 2 }
```

Tidak perlu mengubah logika UI di dalam CounterScreen.

- c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

- Tidak ada perubahan yang diperlukan pada CounterScreen.
- CounterScreen tetap:

```
@Composable  
fun CounterScreen(  
    modifier: Modifier, number: Int, label: String,  
    onButtonClick: () -> Unit  
) {  
    Column(modifier) {  
        Text(  
            text = "$number", fontSize = 72.sp  
        )  
  
        Button(  
            onClick = onButtonClick  
        ) {  
            Text(text = "$label")  
        }  
    }  
}
```

Perubahan kebutuhan hanya dilakukan pada state dan event yang dikelola oleh pemanggil CounterScreen.

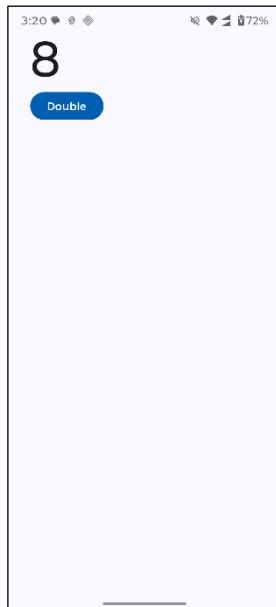
- 7) Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan *state hoisting* dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?
- State hoisting memberikan manfaat karena pengelolaan state dan logika event dipisahkan dari tampilan UI. Pada percobaan ini, ketika kebutuhan aplikasi berubah dari tombol Increment menjadi Double, perubahan cukup dilakukan pada state, label, dan lambda onButtonClick di dalam onCreate(). Fungsi CounterScreen tidak perlu diubah karena composable tersebut hanya menerima data dan event melalui parameter. Dengan demikian, perubahan kebutuhan fungsional dapat dilakukan dengan lebih mudah tanpa harus mengubah struktur UI yang sudah dibuat.

2. Perubahan Konfigurasi

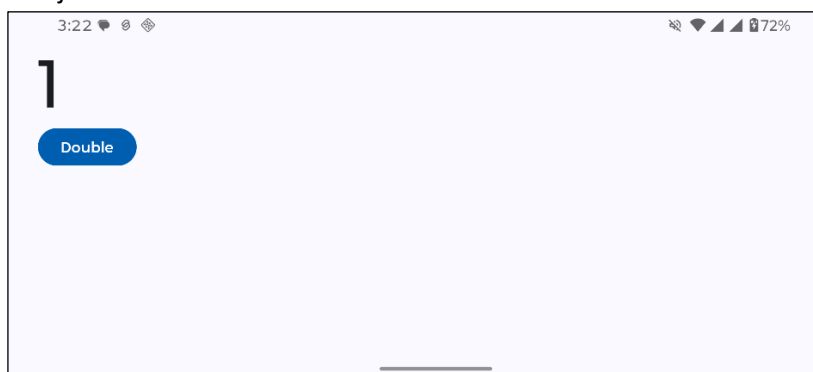
Aktivitas Percobaan

- 1) Jalankan program, dengan memastikan posisi perangkat pada posisi **portrait**, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.

Setelah diklik 3 kali:



- 2) Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi **portrait** menjadi **landscape**, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.
 - Angka sebelumnya 8, setelah perangkat diputar menjadi **landscape**, angka kembali menjadi 1



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?
 - Angka yang ditampilkan kembali menjadi: 1.
Hal tersebut terjadi karena variabel menggunakan **remember** dengan kode: `var number by remember { mutableStateOf(1) }`. **remember** dapat mempertahankan state ketika terjadi recomposition, tetapi tidak mempertahankan state ketika Activity dibuat ulang akibat perubahan konfigurasi, seperti rotasi layar. Ketika Activity dibuat kembali, state number diinisialisasi lagi dengan nilai awal 1.

Modul menjelaskan bahwa `remember` tidak mempertahankan nilai state ketika terjadi perubahan konfigurasi.

Ubah bagian kode program di dalam fungsi `onCreate()` seperti berikut ini:

```
var number by remember { mutableStateOf(1) }
```

menjadi:

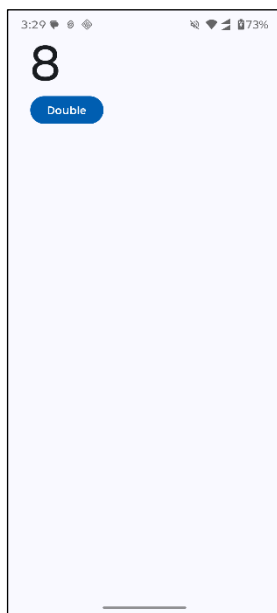
```
var number by rememberSaveable { mutableStateOf(1) }
```

Lakukan *import dependency* atas perubahan konstruk `remember` menjadi `rememberSaveable` yang dilakukan (dengan memposisikan kursor pada konstruk `rememberSaveable` tersebut dan melakukan refactoring dengan menekan kombinasi tombol `Alt-Enter` di jendela *editor* Android Studio).

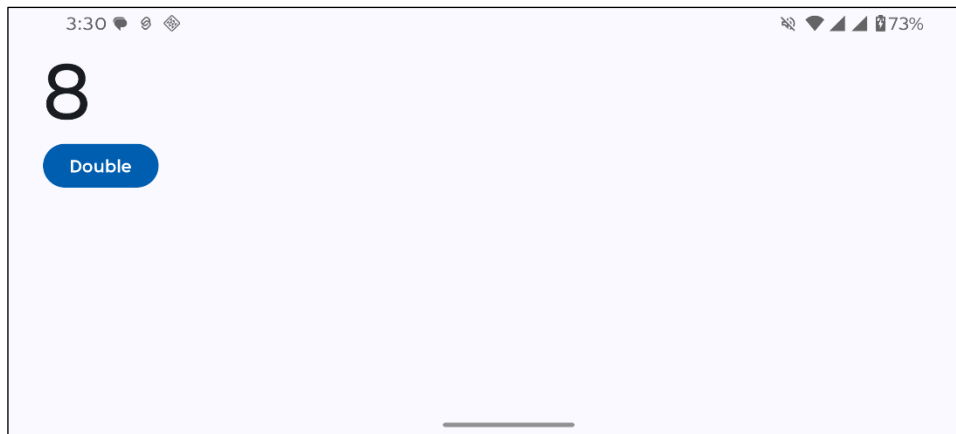
Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi **portrait**, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel `number` saat program pertama kali dijalankan.

Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.

- Setelah tombol Double diklik 3 kali, angka yang ditampilkan adalah 8.



- 3) Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi *landscape*.
 - Angka yang ditampilkan adalah 8.



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?
 - Angka yang ditampilkan tetap 8. Nilai tetap 8 karena variabel number sekarang menggunakan rememberSaveable:


```
var number by rememberSaveable { mutableStateOf(1) }
```

 rememberSaveable tidak hanya mempertahankan state selama recomposition, tetapi juga dapat menyimpan state sehingga nilainya dapat dipulihkan ketika terjadi perubahan konfigurasi seperti rotasi layar. Oleh karena itu, ketika Activity dibuat ulang setelah perangkat diputar ke landscape, nilai number yang sebelumnya 8 dapat dipulihkan kembali dan tidak kembali ke nilai awal.
 - c. Apa fungsi dari konstruk `rememberSavable` pada kode program Android yang membedakannya dari konstruk `remember`?
 - `rememberSaveable` digunakan untuk mempertahankan nilai state tidak hanya ketika terjadi recomposition, tetapi juga ketika terjadi perubahan konfigurasi, seperti rotasi perangkat yang menyebabkan Activity dibuat ulang. Perbedaannya: jika menggunakan `remember`, state pada configuration change dapat hilang, sedangkan jika menggunakan `rememberSaveable`, state dapat dipulihkan.
- 4) Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSavable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!
- Salah satu contoh penggunaan `rememberSaveable` adalah pada form pengisian data pengguna, misalnya form registrasi yang memiliki field nama, email, atau alamat. Jika pengguna sudah mengetik sebagian data kemudian perangkat diputar sehingga terjadi perubahan konfigurasi, isi field tersebut dapat dipertahankan menggunakan `rememberSaveable`. Manfaatnya adalah pengguna tidak perlu mengisi ulang data yang sudah dimasukkan sehingga pengalaman penggunaan aplikasi menjadi lebih baik.

3. LaunchedEffect

Aktivitas Percobaan 1

- 1) Jalankan program,
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?
 - Layar menampilkan angka 1, tombol Double, dan sebuah Snackbar dengan pesan: "Number 1 is shown!"

- b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?
 - Snackbar langsung muncul ketika aplikasi pertama kali dijalankan karena `LaunchedEffect(number)` dijalankan ketika `composable` pertama kali masuk ke komposisi. Nilai awal `number` adalah 1, sehingga `LaunchedEffect(1)` menjalankan kode: `snackbarHostState.showSnackbar("Number $number is shown!")`
 - c. Apa yang memicu pesan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?
 - Snackbar dipicu oleh `LaunchedEffect(number)`. Bagian kode yang bertanggung jawab adalah:


```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar("Number $number is shown!")
}
```

`number` menjadi key dari `LaunchedEffect`. Ketika `number` berubah, `LaunchedEffect` akan dijalankan kembali dan menampilkan Snackbar dengan nilai `number` yang terbaru.
- 2) Klik button "Double" satu kali,
- a. Apa yang terjadi?
 - Angka yang ditampilkan berubah dari 1 menjadi 2 dan snackbar kembali muncul dengan pesan: "Number 2 is shown!"
 - b. Mengapa hal tersebut terjadi? Jelaskan!
 - Ketika tombol Double diklik, lambda `onButtonClick = { number *= 2 }` dijalankan sehingga nilai state `number` berubah dari 1 menjadi 2. Perubahan state menyebabkan recomposition. Karena `number` merupakan key dari `LaunchedEffect`, perubahan nilai tersebut menyebabkan `LaunchedEffect` dijalankan kembali. Akibatnya `snackbarHostState.showSnackbar()` dipanggil kembali dan menampilkan pesan "Number 2 is shown!".

Aktivitas Percobaan 2

- 1) Jalankan program,
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?
 - Pada saat pertama kali program dijalankan, nilai awal variabel `number` adalah 1 sehingga layar menampilkan angka 1 dan tombol Double. Snackbar juga menampilkan pesan "Number 1 is shown!".
 - b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?
 - Setelah menunggu sekitar dua detik, nilai `number` berubah menjadi 996, snackbar menampilkan pesan "Number 996 is shown!".
 - c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?
 - Perubahan angka dipicu oleh `LaunchedEffect(Unit)` pada kode:


```
LaunchedEffect(Unit) {
    number = Repo.getData()
}
```

Fungsi `Repo.getData()` menunggu selama dua detik menggunakan `delay(2000)`, kemudian mengembalikan angka acak yang diberikan ke variabel `number`. Setelah `number` berubah, `LaunchedEffect(number)` dijalankan kembali karena nilai `key` `number` berubah. Effect tersebut memanggil:

```
snackbarHostState.showSnackbar(  
    "Number $number is shown!"  
)
```

sehingga `Snackbar` menampilkan pesan berdasarkan nilai `number` terbaru.

- 2) Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,
 - a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?
 - `LaunchedEffect(state)` menggunakan sebuah `state` sebagai `key`. Effect akan dijalankan ketika `composable` pertama kali masuk ke komposisi dan akan dijalankan kembali apabila nilai `state` yang menjadi `key` berubah. Sementara itu, `LaunchedEffect(Unit)` menggunakan `Unit` sebagai `key` yang tidak berubah. Effect akan dijalankan ketika `composable` pertama kali masuk ke dalam komposisi dan tidak dijalankan kembali akibat perubahan `state` lain.
 - b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?
 - `LaunchedEffect(state)` dijalankan saat `composable` pertama kali masuk ke komposisi dan dijalankan kembali ketika nilai `state` sebagai `key` berubah. `LaunchedEffect(Unit)` dijalankan saat `composable` pertama kali masuk ke komposisi dan tidak dijalankan kembali hanya karena terjadi perubahan pada `state` lain.